Image Denoising and Inpainting using Convolutional Neural Network

video at:
https://youtu.be/LO7M_waVvDc

國立臺北大學 411086018 梁博森
國立臺北大學 411086034 羅廣翔
國立臺北大學 411079059 陳文澤

Abstract

Image denoising is crucial for enhancing image quality, especially in challenging conditions like low light. This project proposes a CNN-based approach to remove noise while preserving important image features. CNN-based methods can outperform traditional techniques by learning complex noise patterns and better handling noise variations. This work expects to improve on the performance of existing methods by utilizing deeper architectures and optimized training techniques.

1. Introduction

1.1. Motivation

Image noise can degrade the quality of images captured under non-ideal conditions (e.g., low light, high ISO, etc.). Removing this noise is essential in fields like photography, medical imaging, or even satellite imagery where image clarity is crucial.

CNN-based image denoising has shown promise due to its ability to model complex noise patterns and learn from large datasets. We aim to eliminate noise and repair image defects through non-traditional methods, rather than relying solely on filters and wavelet transforms.

1.2. Problem Definition

The challenge is to reduce different types of image noises while maintaining essential image features. Traditional denoising methods may blur edges or fail to generalize well to different noise types. The goal of this project is to develop a CNN-based model capable of learning noise patterns automatically and applying the appropriate denoising strategy for different noise types.

1.3. Existing Solutions

Traditional methods, such as average filtering and wavelet transforms, offer basic denoising but may not adapt well to different types of noise. Advanced methods like DnCNN (Denoising CNN) use deep learning to improve denoising but still face challenges with certain noise types. Our approach uses one model to recognize what type of noises an input image has, and three separate models for three different noise types, each trained to perform denoising specifically for one kind of noise.

1.4. Expected Solution

The project expects that the CNN models for denoising will outperform traditional methods by achieving better noise reduction and preserving image details. Additionally, the noise classifier will enhance the system by automatically identifying the noise type in any given image and applying the appropriate model for denoising, ensuring better results for diverse images.

2. Method

2.1. CNN Architecture

We employed a CNN-based architecture for denoising that can process images with Gaussian, Cosine, and Thermal noise. The architecture includes:

- Input Layer: Images are input as grayscale images of size 321×321 pixels, preprocessed by adding one of three types of noise (Gaussian, Cosine, or Thermal).
- Convolutional Layers: These layers apply various filters to capture noise patterns and image features.
- Activation Functions: ReLU activations introduce non-linearity, allowing the model to learn complex noise patterns.

- Fully Connected Layers: After feature extraction, the model applies fully connected layers to generate the final denoised output.

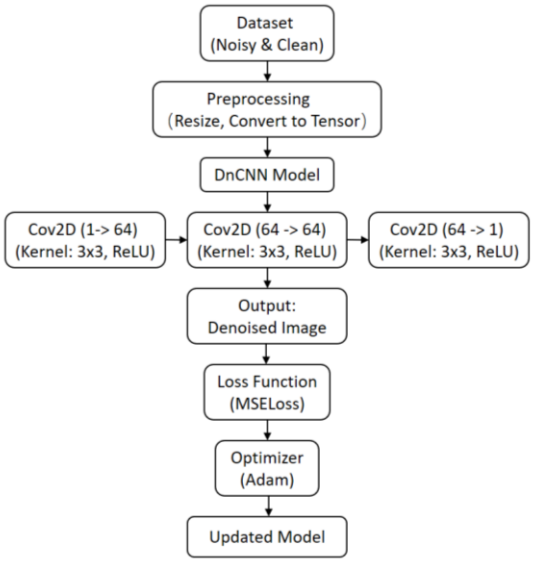

Figure 1. This architecture is designed to learn noise patterns and improve the denoising results through a combination of deep learning techniques.

Additionally, we implemented a noise classifier that uses a similar CNN-based architecture. This classifier is trained to identify whether the image has Gaussian, Cosine, or Thermal noise. The classifier's output determines which of the three denoising models should be applied.

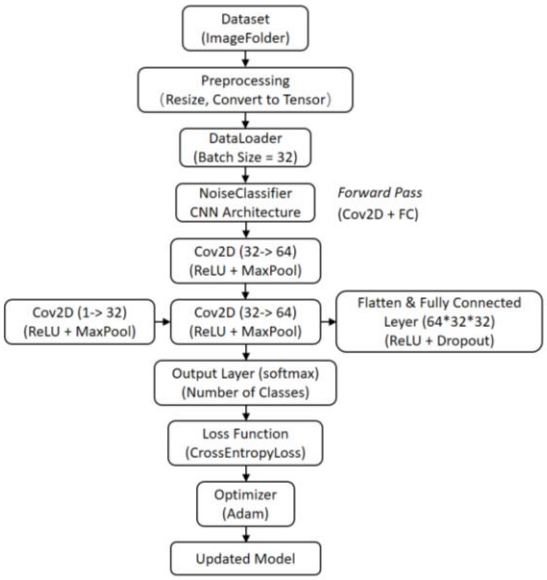

Figure 2: This architecture is designed to classify images based on noise patterns (e.g., Gaussian, Cosine, Thermal) and improve denoising performance using deep learning techniques. The model learns to identify and classify noise through a series of convolutional and fully connected layers.

2.2. Training Parameters

- Learning Rate: A learning rate of 0.001 is used, and adjusted during training for optimization.
- Batch Size: We used a batch size of 32.
- Epochs: The models are trained over 10 epochs.

2.3. Loss Function

The network is trained to minimize the Mean Squared Error (MSE) between the denoised output and the ground truth clean image. This helps ensure that the denoised image is as close as possible to the clean reference image.

The MSE between two images I_A (the clean image) and I_B (the denoised image) is calculated as follows:

$$MSE = \frac{1}{m \cdot n} \sum_{i=1}^{m} \sum_{j=1}^{n} \left(I_A(i,j) - I_B(i.j)\right)^2 \qquad (1)$$

where:

- $I_A(i,j)$ and $I_B(i,j)$ are the pixel values at position (i,j) of the clean and denoised images, respectively.
- m and n are the height and width of the image.
- The sum is performed over all pixels in the image.

2.4. Data Directory Structure

The dataset contains three subfolders based on the type of noise: Gaussian, Cosine, and Thermal noise. Each subfolder contains 400 images, providing a total of 1200 images for training. These three subfolders are placed within the same parent directory to facilitate efficient loading and processing of the images during model training.

2.5. Data Handling

The dataset is organized further into training and validation sets, with 90% of the images used for training and 10% reserved for validation. This split allows continuous evaluation of the model's performance throughout the training process.

2.6. Dataset

We collected various images from the internet. These raw images will be processed by adding different intensities of Gaussian noise, Cosine noise, and Thermal noise.

2.7. Training Strategy

Training is conducted using Adam optimizer, with techniques like learning rate scheduling and data augmentation (rotation, scaling, etc.) to improve generalization.

3. Expected Results

3.1. Performance Improvements

The proposed CNN model is expected to achieve lower Mean Squared Error (MSE) scores compared to traditional methods like average filtering.

3.2. Visual Quality

Figure 3. Comparison of denoising performance. Left: Input image with Gaussian noise. Right: Denoised image using the proposed CNN-based model, showing improved detail preservation and noise reduction.

Visually, the denoised images will have sharper edges and better-preserved textures, with minimal artifacts and blurring. The model should generalize well to different types of noises.

3.3. Computational Efficiency

By utilizing optimizations like batch normalization and residual learning, we expect the model to be computationally more efficient than existing deep learning models, allowing for faster inference on high-resolution images.

4. Experiment

4.1. Dataset Preparation

For this project, we utilize the Berkeley Segmentation Dataset and Benchmark (BSDS500) as our primary data source. Additionally, we will add images from the Open Images Dataset V7 provided by Google to further enhance the diversity of our training and testing dataset.

Dataset Selection

- BSDS500: This dataset consists of 500 images that capture a variety of textures and structures, making them ideal for training denoising models.
- Open Images Dataset V7: This dataset contains millions of labeled images across various categories, which is also suitable for model training.

Noise Simulation

To create a suitable training dataset, we introduce three types of noise into clean images using the **skimage** library. **Gaussian noise** is added to simulate random fluctuations in pixel values; **Cosine noise** is used to replicate periodic disturbances or interference. And finally, **Thermal noise** is introduced to mimic the random fluctuations caused by thermal variations in the image sensor. These different types of noise are applied to distinct groups of clean images, providing a diverse range of scenarios for the model to learn from.

Data Loading and Preprocessing

Images with different noise types are loaded from the specified directories, resized to a consistent shape of (321, 321) pixels, and converted to a floating-point representation for processing. The choice of 321×321 pixels is primarily due to memory constraints and GPU performance limitations, allowing for efficient processing during training.

4.2. Experimental Setup

The experimental setup for training the CNN-based image denoising model includes several key components that ensure effective training and evaluation of the model.

Hardware and Software Configuration

- Hardware: Initially, we used Google Colab to train our model due to limitations with the local CPU and the inability to effectively utilize the GPU on our device.

However, we have now successfully set up our device to utilize the GPU for training. This was achieved by configuring NVIDIA CUDA, which allows us to harness the power of the GPU, significantly speeding up the training process and improving data processing efficiency.

- Software: The model implementation is now based on the PyTorch framework, which is widely recognized for its flexibility and performance in deep learning tasks. Previously, we were planning to use Keras with TensorFlow, but we shifted to PyTorch due to its superior support for custom model architectures and its seamless integration with GPU acceleration. The code is written in Python, with additional libraries such as NumPy for data handling, and skimage for image preprocessing.

Comparison Procedure:

The comparison will focus on evaluating the performance of the DnCNN model across different types of noise cancellation, including Gaussian, Cosine, and Thermal noise. Each noise type will be handled by a separate DnCNN model, and the performance will be assessed using the defined metrics. The results will be analyzed to highlight the improvements in denoising quality for each type of noise, ensuring a comprehensive understanding of the model's effectiveness in handling diverse noise conditions.

4.3. Results Analysis

In this section, we evaluate the performance of various models in denoising images corrupted by three types of noise: Gaussian, Cosine, and Thermal. These noise types were added to a set of 68 images, and separate models were applied for denoising each type. The Mean Squared Error (MSE) was calculated and compared across these images to assess model performance.

Table 1. Comparison of MSE for Different Methods.

Model	Clean vs. Denoised Gaussian	Clean vs. Denoised Cosine	Clean vs. Denoised Thermal
MSE	0.0021606	0.0002579	0.0018218

Figure 4. Original, noisy (Gaussian noise), and denoised (using DnCNN) images. The denoised image shows a slight blur but significant noise reduction.

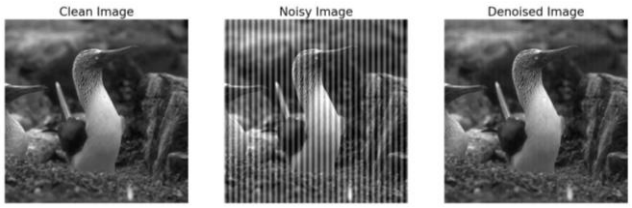

Figure 5. Image with Cosine noise before and after denoising using DnCNN. The denoised image retains key details while reducing noise.

Figure 6. Image with Thermal noise before and after denoising. The denoised image improves clarity and sharpness despite slight blurring.

Our classifier model, designed to distinguish between different noise types, misclassifies Gaussian noise as Thermal noise. This issue arises because thermal noise is a special case of Gaussian noise, both following similar statistical properties. Despite this, the model performs well in terms of training accuracy, as shown in Figure 7, indicating effective learning from the data. However, further improvements in training data and feature extraction are needed to refine the model’s ability to differentiate between Gaussian and thermal noise.

```
加載樣本數量：1200
類別名稱：['noisy', 'noisy_cos', 'noisy_thermal']
Epoch [1/10], Loss: 0.7738, Accuracy: 63.58%
Epoch [2/10], Loss: 0.3851, Accuracy: 79.42%
Epoch [3/10], Loss: 0.2789, Accuracy: 87.33%
Epoch [4/10], Loss: 0.2209, Accuracy: 91.33%
Epoch [5/10], Loss: 0.1848, Accuracy: 92.75%
Epoch [6/10], Loss: 0.1725, Accuracy: 93.92%
Epoch [7/10], Loss: 0.1771, Accuracy: 93.00%
Epoch [8/10], Loss: 0.1590, Accuracy: 94.67%
Epoch [9/10], Loss: 0.1454, Accuracy: 95.25%
Epoch [10/10], Loss: 0.1578, Accuracy: 93.83%
模型訓練完成並儲存！
```

Figure 7. Training Accuracy of Classifier.pth Over Epochs.

Key Observations:

- Gaussian Noise Denoising: The denoised image reduces Gaussian noise, though slightly blurry, resulting in a cleaner, sharper image compared to the noisy version.
- Cosine Noise Denoising: DnCNN successfully reduces Cosine noise while preserving essential image details, resulting in a cleaner, sharper output.
- Thermal Noise Denoising: DnCNN improves image quality by reducing thermal noise, providing a clearer and sharper appearance, despite some blurriness.
- Classifier: While the classifier distinguishes Cosine noise well, it misclassifies Gaussian noise as Thermal noise due to the statistical overlap between these two noise types.

Possible Reasons:

1. Statistical Overlap: Both Gaussian and thermal noise share similar statistical characteristics, making them difficult to distinguish in the feature space. Thermal noise is essentially a special case of Gaussian noise, contributing to the misclassification.
2. Limited Training Data: The model may not have enough diverse examples, especially for Gaussian and Thermal noise at varying intensities, which hinders its ability to learn clear distinctions.
3. Feature Extraction Limitations: The model relies on spatial domain features, which may not fully capture the nuances between these two similar noise types. More advanced techniques, such as frequency-domain features, could help.

4.4. Limitations and Future Work

To enhance the classifier's ability to differentiate between Gaussian and thermal noise, we propose several improvements:

1. Better Feature Extraction: Incorporating more sophisticated feature extraction techniques, such as frequency-domain features (e.g., Fourier or wavelet transforms), could help capture subtle differences between noise types that are not easily detectable in the spatial domain.
2. Augmented Training Data: Expanding the dataset to include a wider variety of noise characteristics, especially by adding more examples with varying levels of Gaussian and thermal noise, could help the model learn to distinguish between the two more effectively.
3. Separate Classification Models: Training separate classifiers for Gaussian and thermal noise could help isolate the differences more clearly, ensuring that each noise type is better represented and learned independently.
4. Model Refinement: Further refinement of the classifier, perhaps by experimenting with more advanced architectures like much deeper Convolutional Neural Networks (CNNs) or integrating attention mechanisms, could improve the classifier's sensitivity to subtle noise differences.

References

[1] Convolutional Neural Network – Wikipedia, available on: https://en.m.wikipedia.org/wiki/Convolutional_neural_network

[2] Methods for image denoising using convolutional neural network: a review, available on: https://link.springer.com/article/10.1007/s40747-021-00428-4

[3] Impact of Traditional and Embedded Image Denoising on CNN-Based Deep Learning, available on: https://www.researchgate.net/publication/374939473_Impact_of_Traditional_and_Embedded_Image_Denoising_on_CNN-Based_Deep_Learning

[4] DENOISING AND ENHANCING NOISY IMAGES USING CNN AND ITERATIVE FILTERING TECHNIQUES, available on: https://www.researchgate.net/publication/382113856_DENOISING_AND_ENHANCING_NOISY_IMAGES_USING_CNN_AND_ITERATIVE_FILTERING_TECHNIQUES